Projet Sokoban

POO – Programmation Orientée Objet en Java | LI2
Semestre 4

Université d'Artois – Faculté des Sciences Jean Perrin

1. Introduction

Ce document présente le travail réalisé dans le cadre du TP n°7 portant sur le développement du jeu Sokoban en Java avec une approche orientée objet. Le projet couvre les étapes 1 à 17 du sujet, soit la totalité du modèle (package modele) et la version jouable en mode texte (package vueTexte). Sokoban est un jeu de réflexion dans lequel un robot doit pousser des caisses sur des emplacements prévus, en ne pouvant ni tirer les caisses ni en pousser plusieurs à la fois.

2. Organisation du projet

Conformément à l'étape 1, le code source est séparé en packages distincts afin de respecter le patron MVC (Modèle / Vue) :

Package / Répertoire Contenu

modele/ Classes métier indépendantes de tout affichage

vueTexte/ Vue et boucle de jeu en mode console

vueGraphique/ Vue graphique Swing (étapes 19+, non traitées ici)

map/ Fichiers de cartes map1.txt, map2.txt, map3.txt

images/ Sprites GIF fournis (utilisés par la vue graphique)

3. Modèle : hiérarchie des éléments (étapes 2 – 3)

L'observation des cartes révèle 7 types de cases : mur, vide, sol, destination, caisse (sur sol), caisse sur destination, robot (sur sol ou sur destination). Les quatre premiers sont des éléments fixes ; les trois derniers résultent de la combinaison d'un élément fixe et d'un occupant (robot ou caisse).

Choix de conception : une classe abstraite Element porte deux booléens aJoueur et aCaisse.

Quatre sous-classes concrètes en héritent :

• Mur – infranchissable, caractère #

• Vide – hors du périmètre jouable, caractère /

• Sol – franchissable, caractère espace

• Destination – franchissable, caractère .

La méthode toString() d'Element calcule dynamiquement le bon caractère d'affichage (@, +, $, *) selon les booléens, ce qui évite de dupliquer cette logique dans chaque sous-classe.

4. La classe Carte (étapes 4, 8 – 10, 13 – 15, 18)

4.1 Attributs principaux

Attribut Type Rôle

grille Element[][] Grille 2D des éléments (indexée [ligne][col])

robot Robot Instance unique du robot, mémorise position et direction

destinations List<int[]> Coordonnées {col, ligne} de chaque destination

lignesOriginales List<String> Lignes brutes du fichier (pour restart – étape 24)

4.2 Lecture du fichier (étapes 8 – 9)

La classe Lecture encapsule la lecture d'un fichier texte via BufferedReader. Elle expose la liste des lignes, le nombre de lignes et la largeur maximale. Carte délègue entièrement cette lecture à Lecture, ce qui respecte le principe de responsabilité unique.

4.3 Déplacement du robot (étape 14)

La méthode deplacer(Direction dir) suit cet algorithme :

• Calcul de la case visée (position robot + dx/dy de la direction)

• Vérification des bornes et du caractère franchissable de la case

• Si une caisse est présente : calcul de la case suivante et vérification qu'elle est libre et franchissable

• Mise à jour des booléens aJoueur / aCaisse sur les éléments concernés

• Mise à jour de la position et de la direction du robot

• Retour d'un booléen indiquant si le mouvement a eu lieu

4.4 Fin de partie (étape 18)

La méthode finDePartie() parcourt la liste des destinations mémorisées à la construction et retourne false dès qu'une destination ne porte pas de caisse. La partie est gagnée si et seulement si toutes les destinations sont couvertes.

5. Direction et Robot (étapes 11 – 12)

Direction est une enum à quatre valeurs (HAUT, BAS, GAUCHE, DROITE), chacune portant ses incréments dx et dy. Cela permet d'écrire les calculs de déplacement de manière uniforme, sans switch dans Carte.

Robot stocke sa colonne, sa ligne et sa dernière Direction (utile pour choisir l'image orientée dans la vue graphique). Ses setters sont appelés exclusivement par Carte, ce qui centralise la logique métier dans le modèle.

6. Vue en mode texte (étapes 16 – 17)

6.1 Classe ModeTexte

La classe ModeTexte possède :

• Le modèle (Carte) comme attribut

• Une HashMap<Character, Direction> associant chaque touche clavier à une direction. Choix AZERTY : z=haut, s=bas, q=gauche, d=droite.

La méthode jouer() boucle tant que finDePartie() retourne false : elle affiche la carte, lit une direction

valide via Outil.lireCaractere(), tente le déplacement et signale tout mouvement impossible.

6.2 Classe SokobanTexte (point d'entrée)

Le main accepte optionnellement le nom du fichier de carte en argument (sinon map/map1.txt par défaut). Il instancie Carte et ModeTexte, affiche la carte initiale pour vérification, puis lance la partie.